باسمه تعالی

آزمایشگاه طراحی سیستم‌های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

تابستان ۱۴۰۳

استاد:

دکتر محسن انصاری

مهندس محمدرضا حسینی

اعضای گروه:

سعید فراتی کاشانی – ۴۰۱۱۰۶۲۹۹

امیرحسین صوری – ۴۰۱۱۰۶۱۸۲

محمدامین عباس‌فر – ۴۰۱۱۰۶۲۱۱

فهرست عناوین

موضوع آزمایش ۳
شرح کلی آزمایش ۳
پیاده‌سازی Stack ۳
ورودی‌ها ٤
خروجی‌ها ٤
طراحی Stack ٥
برنامه‌ی محک (TestBench) ۷

موضوع آزمایش

توصیف رفتاری (Stack)

شرح کلی آزمایش

۲-۵ آزمایش چهارم: توصیف رفتاری

یک پشته با عمق ۸ و پهنای ٤ بیت طراحی کنید که دارای ورودی‌ها و خروجی‌های زیر باشد :

Inputs:	Clk	Clock signal
	RstN	Reset signal
	Data_In	4-bit data into the stack
	Push	Push Command
	Pop	Pop Command
Outputs:	Data_Out	4-bit output data from stack
	Full	Full=1 indicates that the stack is full
	Empty	Empty=0 indicates that the stack is empty

برای پیاده‌سازی استک مورد نظر به صورت توصیف رفتاری (Behavioral Model) باید از مقداردهی رجیسترها (در وریلاگ) در بلاک‌های initial و always استفاده کنیم.

پیاده‌سازی Stack

برای پیاده‌سازی استک، کافی است یک حافظه داشته‌باشیم (که همان استک یا پشته‌ی ماست) و به کمک یک stack pointer (sp) به اولین خانه‌ی حافظه اشاره کنیم تا بتوانیم عملیات push و pop را اعمال کنیم.

ورودی‌ها

Clk: سیگنال کلاک

RstN: سیگنال ریست که به صورت پایین‌رونده اعمال می‌شود.

Data_In: داده‌ی ورودی که ۴ بیتی است و قرار است در اولین خانه‌ی خالی استک یا همان‌جایی که sp به آن اشاره می‌کند، قرار بگیرد.

Push: سیگنال push که در صورت فعال بودن، عملیات push را با دیتای Data_In انجام می‌دهد.

Pop: سیگنال pop که در صورت فعال بودن، عملیات pop را انجام می‌دهد؛ یعنی داده‌ای که sp به آن اشاره می‌کند را در Data_Out خروجی می‌دهد.

خروجی‌ها

Data_Out: در صورت اعمال pop، داده در این خروجی قرار می‌گیرد.

Full: در صورتی که استک پر شده‌باشد، فعال می‌شود.

Empty: در صورتی که استک خالی شده‌باشد، فعال می‌شود.

طراحی Stack

کد وریلاگ مربوط به این استک به پیوست ارسال می‌شود؛ آن را می‌توانید در زیر ببینید:

```
module Stack (
    input Clk, RstN, Push, Pop, input [3:0] Data_In,
    output reg [3:0] Data_Out, output reg Full, Empty);

    parameter DEPTH = 8;
    parameter WIDTH = 4;

    reg [WIDTH-1:0] stack [0:DEPTH-1];
    reg [3:0] sp;

    always @(posedge Clk or negedge RstN) begin
        if (!RstN) begin
            sp <= 0;
            Full <= 0;
            Empty <= 1;
        end else begin
            if (Push && !Full) begin
                stack[sp] <= Data_In;
                sp <= sp + 1;
            end
            if (Pop && !Empty) begin
                sp <= sp - 1;
                Data_Out <= stack[sp - 1];
            end
        end

        Full <= (sp == DEPTH);
        Empty <= (sp == 0);
    end

endmodule
```

کد ماژول Stack را بخش به بخش بررسی می‌کنیم:

در ابتدا ورودی‌ها و خروجی‌ها تعریف شده‌اند. ورودی‌های Clk، RstN، Push، Pop، Full و Empty تک‌بیتی هستند. ورودی Data_In و Data_Out چون داده‌ها را نگه می‌دارند، ۴ بیتی هستند.

در ادامه پارامترهای DEPTH و WIDTH مطابق خواسته‌ی سوال برابر ۸ و ۴ مقداردهی شده‌اند. این دو پارامتر به ترتیب تعداد کلمات استک و تعداد بیت‌های هر داده‌ی استک را نشان می‌دهند.

حالا stack را می‌سازیم؛ که آرایه‌ای به طول DEPTH (۸) از داده‌های WIDTH (۴) بیتی است. رجیستر sp نیز stack pointer است؛ برای اشاره به ۸ خانه (با شماره‌های ۰ تا ۷) کافی است ۳ بیتی باشد، ولی برای

حالتی که استک پر می‌شود و مقدار sp برابر خود ۸ می‌شود، برای این‌که به مشکل مقایسه با صفر نخوریم، آن را ۴ بیتی در نظر می‌گیریم.

بلاک always، در صورت بالا رفتن Clk یا پایین رفتن RstN تریگر می‌شود. حالا به منطق موجود در این بلاک می‌پردازیم:

اگر RstN اعمال شده‌باشد (صفر باشد)، استک خالی می‌شود؛ یعنی sp به خانه‌ی اول اشاره می‌کند (صفر می‌شود) و در نتیجه‌ی آن مقدار خروجی‌های Full و Empty به ترتیب ۰ و ۱ می‌شوند.

در حالت معمولی (لبه‌ی بالارونده‌ی کلاک) نیز اگر سیگنال push فعال باشد، در صورتی که استک پر نشده باشد (یعنی Full برابر صفر باشد)، دیتای ورودی را در محل sp می‌نویسیم و sp را یک واحد افزایش می‌دهیم؛ اگر سیگنال pop فعال باشد، در صورتی که استک خالی نباشد، sp یک واحد کاهش می‌یابد و مقداری که در سر استک بود را خروجی می‌دهیم.

در نهایت نیز مقدار Full و Empty بر اساس مقدار sp آپدیت می‌شوند.

برنامه‌ی محک (TestBench)

```
module TB;
    reg Clk, RstN, Push, Pop;
    reg [3:0] Data_In;

    wire [3:0] Data_Out;
    wire Full, Empty;

    Stack stack (Clk, RstN, Push, Pop, Data_In, Data_Out, Full, Empty);

    always #5
        Clk = ~Clk;

    initial begin
        Clk = 0; RstN = 0; Data_In = 0; Push = 0; Pop = 0;

        #15 RstN = 1;

        #10 Data_In = 4'b0001; Push = 1;
        #10 Data_In = 4'b0010;
        #10 Data_In = 4'b0011;
        #10 Data_In = 4'b0100;
        #10 Data_In = 4'b0101;
        #10 Data_In = 4'b0110;
        #10 Data_In = 4'b0111;
        #10 Data_In = 4'b1000;
        #10 Push = 0;

        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;
        #10 Pop = 1; #10 Pop = 0;

        #50 $finish;
    end

    initial begin
        $monitor("Time: %0t | RstN: %b | Data_In: %b | Push: %b | Pop: %b | Data_Out: %b | Full: %b | Empty: %b",
                 $time, RstN, Data_In, Push, Pop, Data_Out, Full, Empty);
    end

endmodule
```

در تست‌بنچ پس از تعریف رجیسترها و سیم‌های مربوطه نمونه‌ای از ماژول Stack می‌سازیم و ورودی‌ها و خروجی‌ها را تنظیم می‌کنیم. سیگنال Clk هر ۵ واحد زمانی عوض می‌شود.

تست‌بنچ به این صورت است که ابتدا سیگنال Empty فعال است؛ سپس اعداد ۱ تا ۸ را در استک push می‌کنیم. پس از push شدن عدد ۱، سیگنال Empty غیرفعال می‌شود؛ پس از push شدن عدد ۸ نیز سیگنال Full یک می‌شود.

حالا شروع به pop کردن اعداد می‌کنیم. پس از اولین pop، سیگنال Full غیرفعال می‌شود؛ سپس این فرایند را ادامه می‌دهیم و به طور کلی اعداد ۸ تا ۱ به ترتیب pop می‌شوند. در انتها نیز سیگنال Empty دوباره فعال می‌شود.

نتیجه‌ی مانیتور تست‌بنچ را در زیر می‌بینید:

```
D:\University\Semester4\DSD Lab\Experiment3>vvp out
Time: 0 | RstN: 0 | Data_In: 0000 | Push: 0 | Pop: 0 | Data_Out: xxxx | Full: x | Empty: x
Time: 5 | RstN: 0 | Data_In: 0000 | Push: 0 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 1
Time: 15 | RstN: 1 | Data_In: 0000 | Push: 0 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 1
Time: 25 | RstN: 1 | Data_In: 0001 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 1
Time: 35 | RstN: 1 | Data_In: 0010 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 45 | RstN: 1 | Data_In: 0011 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 55 | RstN: 1 | Data_In: 0100 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 65 | RstN: 1 | Data_In: 0101 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 75 | RstN: 1 | Data_In: 0110 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 85 | RstN: 1 | Data_In: 0111 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 95 | RstN: 1 | Data_In: 1000 | Push: 1 | Pop: 0 | Data_Out: xxxx | Full: 0 | Empty: 0
Time: 105 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: xxxx | Full: 1 | Empty: 0
Time: 115 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 1000 | Full: 1 | Empty: 0
Time: 125 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 1000 | Full: 0 | Empty: 0
Time: 135 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0111 | Full: 0 | Empty: 0
Time: 145 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0111 | Full: 0 | Empty: 0
Time: 155 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0110 | Full: 0 | Empty: 0
Time: 165 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0110 | Full: 0 | Empty: 0
Time: 175 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0101 | Full: 0 | Empty: 0
Time: 185 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0101 | Full: 0 | Empty: 0
Time: 195 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0100 | Full: 0 | Empty: 0
Time: 205 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0100 | Full: 0 | Empty: 0
Time: 215 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0011 | Full: 0 | Empty: 0
Time: 225 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0011 | Full: 0 | Empty: 0
Time: 235 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0010 | Full: 0 | Empty: 0
Time: 245 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0010 | Full: 0 | Empty: 0
Time: 255 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 1 | Data_Out: 0001 | Full: 0 | Empty: 0
Time: 265 | RstN: 1 | Data_In: 1000 | Push: 0 | Pop: 0 | Data_Out: 0001 | Full: 0 | Empty: 1
TB.v:63: $finish called at 315 (1s)
```